

Atividade --- Projeto nassauTickets

Nesta atividade, o grupo deverá desenvolver e organizar o projeto **nassauTickets**, um Sistema de Controle de Atendimento para um Laboratório de Análises Clínicas. A atividade tem como objetivo consolidar os conhecimentos de desenvolvimento Web, organização de projetos, Git/GitHub, documentação, React e integração entre as partes de uma aplicação.

O sistema deverá atender aos requisitos definidos no documento do projeto, incluindo o controle de emissão, fila, chamada e atendimento de senhas, além das funcionalidades e regras de negócio descritas neste documento. O projeto deverá ser desenvolvido em grupo e mantido em um repositório público no GitHub.

1. Criação do repositório

Cada grupo deverá criar no GitHub um repositório chamado:

```
nassauTickets
```

O repositório deverá possuir obrigatoriamente:

- Visibilidade pública;
- Licença MIT;
- `.gitignore` apropriado para projetos Node.js;
- `README.md` na raiz;
- Todos os integrantes como membros do repositório.

O Scrum Master será responsável pela criação do repositório e pela atribuição de acesso aos demais integrantes.

2. Trabalho em grupo

A atividade deverá ser desenvolvida em grupos de no máximo 6 participantes.

Todos os integrantes deverão ser membros do repositório **nassauTickets**.

No `README.md` deverá existir obrigatoriamente uma seção chamada:

```
## Membros
```

Essa seção deverá apresentar uma lista ou tabela contendo o nome, matrícula e papel de cada participante.

Os papéis disponíveis são:

- **Scrum Master:** deverá existir apenas um por grupo e será responsável pela criação e organização do repositório e pela coordenação das atividades relacionadas ao projeto.

- **Documentador:** poderá haver mais de um e será responsável pela produção e organização da documentação.
- **Desenvolvedor:** poderá haver mais de um e será responsável pela implementação e evolução do código.
- **Testador:** poderá haver mais de um e será responsável pela verificação do funcionamento, identificação de problemas e validação das funcionalidades.

Exemplo:

Membros

Nome	Matrícula	Papel
Nome do aluno	123456	Scrum Master
Nome do aluno	234567	Documentador
Nome do aluno	345678	Desenvolvedor
Nome do aluno	456789	Testador

A composição da equipe e os papéis deverão ser mantidos durante a atividade.

3. Estrutura obrigatória do repositório

A raiz do repositório deverá possuir obrigatoriamente a seguinte estrutura, em ordem alfabética:

```
nassauTickets/
├── backend/
├── docs/
│   ├── branding/
│   ├── mer/
│   ├── mockups/
│   ├── models/
│   │   └── uml/
│   └── requirements/
├── frontend/
├── .gitignore
├── LICENSE
└── README.md
```

Os diretórios deverão existir no GitHub mesmo quando estiverem vazios.

Como o Git não versiona diretórios vazios, deverá ser utilizado um arquivo **.gitkeep** dentro de cada diretório que não possuir outros arquivos.

Exemplo:

```
docs/
├── branding/
```

```

├── .gitkeep
├── mer/
│   ├── .gitkeep
│   ├── mockups/
│   │   ├── .gitkeep
│   │   ├── models/
│   │   │   ├── uml/
│   │   │   │   ├── .gitkeep
│   │   │   └── requirements/
│   │   │       ├── .gitkeep

```

O **.gitkeep** deverá ser utilizado apenas quando necessário para manter um diretório vazio no Git.

4. Organização dos artefatos

A pasta **docs/** deverá concentrar os artefatos de documentação do projeto.

docs/branding/

Deverá conter os elementos relacionados à identidade visual do sistema, quando produzidos pelo grupo.

docs/mer/

Deverá conter o Modelo Entidade-Relacionamento do sistema e seus respectivos artefatos.

docs/mockups/

Deverá conter os mockups e protótipos das telas do sistema.

docs/models/uml/

Deverá conter os diagramas UML produzidos pelo grupo.

docs/requirements/

Deverá conter os requisitos funcionais, requisitos não funcionais, regras de negócio e demais documentos relacionados aos requisitos do sistema.

A documentação deverá contemplar, conforme aplicável ao projeto, aspectos de segurança, disponibilidade, auditoria, desempenho, concorrência, LGPD e acessibilidade.

5. Sistema nassauTickets

O projeto consiste em um sistema de controle de atendimento de um Laboratório de Análises Clínicas.

O sistema deverá trabalhar com três agentes:

- **AS --- Agente Sistema:** executa as ações do sistema, comunica-se com banco de dados e demais infraestruturas, emite senhas, atualiza o painel e responde aos comandos dos demais agentes.
- **AA --- Agente Atendente:** responsável por chamar o próximo cliente e realizar o atendimento no guichê.

- **AC --- Agente Cliente:** emite sua senha por meio do totem e aguarda a chamada no painel.

O sistema deverá trabalhar com três tipos de senha:

- **SP --- Senha Prioritária**
- **SG --- Senha Geral**
- **SE --- Senha para retirada de Exames**

6. Regras de atendimento

As regras de priorização deverão ser implementadas conforme a especificação do projeto:

```
[SP] → [SE|SG] → [SP] → [SE|SG]
```

A cada novo atendimento deverá ser considerada uma nova senha SP, quando houver uma senha desse tipo na fila. Em seguida, deverá ser considerada uma senha SE, quando disponível, e então uma senha SG.

As prioridades deverão respeitar as seguintes regras:

- SP possui maior prioridade;
- SG possui menor prioridade;
- SE possui atendimento operacional especial e deverá ser chamada após uma SP, quando disponível;
- qualquer guichê poderá atender qualquer tipo de senha;
- caso uma fila esteja vazia, o sistema deverá decidir o próximo atendimento seguindo as regras de prioridade;
- uma senha que não for atendida após duas chamadas deverá ser considerada abandonada;
- aproximadamente 5% das senhas emitidas deverão ser consideradas como não atendidas, conforme a regra apresentada na especificação.

O expediente deverá ocorrer das **7h às 17h**. Os atendimentos iniciados deverão ser concluídos e encerrados pelo atendente. Ao final do expediente, senhas que permanecerem na fila deverão ser descartadas.

7. Numeração das senhas

A numeração deverá seguir o padrão:

```
YYMMDD-PPSQ
```

Onde:

- **YY** = ano da emissão com dois dígitos;
- **MM** = mês com dois dígitos;
- **DD** = dia do mês com dois dígitos;
- **PP** = tipo da senha com dois caracteres;
- **SQ** = sequência da senha por prioridade, com três dígitos e reinício diário.

8. Painel de chamadas

O painel deverá apresentar as **5 últimas senhas chamadas**.

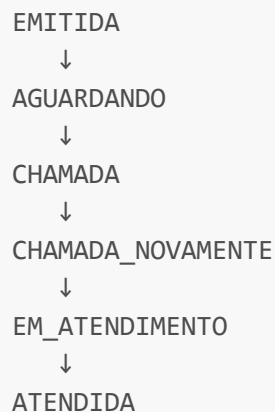
A próxima senha não deverá ser exibida no painel, pois uma nova senha poderá ser emitida entre a finalização de um atendimento e o acionamento do próximo atendimento pelo agente atendente.

O sistema deverá permitir que o atendente:

- chame uma nova senha;
- inicie o atendimento;
- finalize o atendimento;
- chame novamente uma senha quando necessário.

9. Máquina de estados

As senhas deverão possuir uma máquina de estados contemplando a sequência:



Também deverá ser possível chegar ao estado:

NÃO_COMPARECEU

quando o cliente não comparecer após as chamadas previstas.

10. Relatórios

O sistema deverá contemplar relatórios diário e mensal contendo:

- quantitativo geral de senhas emitidas;
- quantitativo geral de senhas atendidas;
- quantitativo de senhas emitidas por prioridade;
- quantitativo de senhas atendidas por prioridade;
- relatório detalhado das senhas;
- tempo médio de atendimento;
- relatório de auditoria.

O relatório detalhado deverá apresentar, quando aplicável:

- número da senha;
- tipo da senha;
- data e hora da emissão;
- data e hora do atendimento;
- guichê responsável.

Para senhas não atendidas, os campos de atendimento deverão permanecer em branco.

O relatório de auditoria deverá registrar:

- atendente;
- guichê;
- senha;
- horário da primeira chamada;
- horário da segunda chamada, se houver;
- horário de início do atendimento;
- horário de finalização.

11. Login e perfis

O sistema deverá possuir login para o agente atendente.

A especificação prevê um único atendente com perfil adicional de gestor, responsável pelos cadastros e relatórios.

O cliente deverá interagir anonimamente por meio do totem.

12. Recursos adicionais

Deverão ser considerados os desafios apresentados na especificação do projeto, especialmente:

- tratamento de concorrência quando dois ou mais atendentes solicitarem a próxima senha praticamente ao mesmo tempo;
- proposta para quantificar e acompanhar o desempenho dos atendimentos;
- áudio durante a chamada, informando prioridade, senha e guichê;
- funcionalidade "**Chamar Novamente**", repetindo o áudio e utilizando a indicação "**Última chamada**";
- comportamento do frontend e do painel diante de falhas do backend ou banco de dados.

13. Tecnologias

O frontend deverá ser desenvolvido com **React**, utilizando os conceitos estudados na disciplina.

A infraestrutura indicada no documento do projeto contempla:

- **MySQL 8.0** para banco de dados;
- **Node.js LTS 22 com Express**, Java 21 com Spring Boot ou Python 3.14 com Flask/FastAPI para backend;
- **React 19** ou Angular 22 para frontend.

Para esta atividade, o desenvolvimento do frontend deverá utilizar React.

A escolha da tecnologia do backend poderá seguir uma das opções indicadas no documento, desde que o grupo mantenha a organização definida e justifique tecnicamente sua escolha no README.

14. Git e branches

O repositório deverá possuir obrigatoriamente, no mínimo:

```
main  
dev
```

A branch `dev` deverá ser utilizada para o desenvolvimento.

Os códigos deverão ser enviados primeiro para `dev` e, posteriormente, integrados à `main` por meio de merge.

O histórico do Git deverá permitir identificar o desenvolvimento realizado na `dev` e o posterior merge para a `main`.

Sempre que possível, utilizem commits pequenos e objetivos, representando etapas reais do desenvolvimento.

Exemplos:

```
chore: cria estrutura inicial do projeto  
docs: adiciona requisitos do sistema  
feat: cria componente de emissão de senha  
feat: implementa fila de atendimento  
feat: implementa painel de chamadas  
fix: corrige regra de prioridade  
docs: atualiza documentação
```

15. Frontend

Todo o código do frontend deverá estar dentro de:

```
frontend/
```

O projeto React deverá ser executável a partir desse diretório.

Exemplo:

```
cd frontend  
npm install  
npm run dev
```

A estrutura interna poderá ser organizada pelo grupo, mas deverá apresentar separação adequada entre componentes, páginas, serviços, recursos e demais elementos necessários à aplicação.

O projeto deverá utilizar componentes React e demonstrar o uso dos conceitos estudados na disciplina, incluindo, conforme a necessidade da aplicação:

- JSX;
- componentes;
- props;
- estado;
- eventos;
- `useState`;
- `useEffect`;
- renderização condicional;
- renderização de listas;
- formulários;
- comunicação assíncrona;
- consumo de APIs REST;
- JSON;
- organização e reutilização de componentes.

16. README.md

O `README.md` deverá permitir que outra pessoa compreenda o projeto e consiga executá-lo.

Deverá conter, no mínimo:

- nome do projeto;
- descrição;
- objetivo;
- tecnologias utilizadas;
- arquitetura ou visão geral do sistema;
- instruções de instalação;
- instruções de execução;
- informações necessárias para configuração;
- identificação dos membros;
- papéis dos membros;
- informações sobre branches;
- demais informações necessárias para utilização do sistema.

A seção de membros deverá obrigatoriamente se chamar:

```
## Membros
```

17. .gitignore

O `.gitignore` deverá ser adequado para projetos Node.js.

Arquivos gerados automaticamente pelas dependências não deverão ser versionados.

Por exemplo:

```
node_modules/
```

O `.gitignore` deverá ser mantido na raiz do repositório.

18. Licença

O repositório deverá possuir a licença MIT no arquivo:

```
LICENSE
```

O arquivo deverá estar na raiz do repositório.

19. Critérios de verificação

Serão verificados:

- repositório `nassauTickets` criado corretamente;
- visibilidade pública;
- licença MIT;
- `.gitignore` para Node.js;
- estrutura obrigatória de diretórios;
- uso de `.gitkeep` quando necessário;
- branches `main` e `dev`;
- desenvolvimento realizado na `dev`;
- merge da `dev` para `main`;
- participação dos integrantes no repositório;
- seção `Membros` no README;
- organização do código;
- utilização de React no frontend;
- implementação das funcionalidades previstas;
- atendimento às regras de negócio;
- documentação e artefatos;
- qualidade e organização dos commits;
- funcionamento e possibilidade de execução do projeto.

20. Entrega pelo Teams

A entrega deverá ser realizada pelo Teams informando a **URL pública do repositório `nassauTickets` no GitHub**.

O repositório deverá estar acessível no momento da avaliação.

O professor deverá conseguir verificar diretamente no GitHub:

- a estrutura dos arquivos;
- o código do projeto;
- a documentação;
- os integrantes;
- as branches;
- o histórico de commits;
- o merge da `dev` para a `main`.

Importante: a atividade não consiste apenas em desenvolver o sistema. O objetivo é demonstrar a capacidade de organizar, documentar, versionar e desenvolver um projeto Web utilizando práticas de desenvolvimento de software e os conhecimentos trabalhados na disciplina.

21. Referência do projeto

Os requisitos funcionais, regras de negócio e desafios apresentados nesta atividade têm como base o documento **Sistema para controle de atendimento**, que especifica o funcionamento do sistema de tickets para o Laboratório de Análises Clínicas, incluindo agentes, tipos de senha, prioridades, horários, relatórios, auditoria, máquina de estados, segurança, disponibilidade, concorrência, LGPD e acessibilidade.